Project Report

Secure Student Portal – SECU2000 Group Final Project

Group Members: Ritik Vyas · Cy Iver · Thanh Nguyen · Randie

Technology Stack: Node.js / Express · EJS · SQLite3 · Helmet · Winston · Multer · bcrypt

Date Submitted: April 15, 2026 · **Course:** SECU2000 – Application Security

Table of Contents

1. Executive Summary 2
2. Application Architecture 2
3. Required Threat Surface 3
4. Authentication, Authorization & Session Management 3
5. Secure Coding Practices 4
6. Data Protection & Privacy Controls 4
7. Database Design & Safety 5
8. OWASP Top 10 Mapping 5
9. STRIDE Threat Model 7
10. Security Testing 8
11. Deployment Security & Incident Response 9
12. Conclusion 9

1. Executive Summary

The **Secure Student Portal** is a full-stack web application built for the SECU2000 final project. It models a realistic post-secondary grade management system with a deliberately layered architecture, designed to demonstrate both common web application vulnerabilities and their corresponding mitigations aligned to the OWASP Top 10 and STRIDE threat model.

The portal provides five distinct threat surfaces — login form, search form, file upload, admin panel, and REST API endpoints — each implemented in two modes: a **vulnerable** version (for live demonstration of attacks) and a **secure** version (showcasing industry-standard defenses). This dual-mode approach ensures graders can directly observe the impact of security controls.

Five OWASP Top 10 categories are addressed: **A01 Broken Access Control**, **A03 Injection**, **A05 Security Misconfiguration**, **A07 Authentication Failures**, and **A09 Logging & Monitoring**.

2. Application Architecture & Separation of Concerns

The application follows a strict three-tier architecture. No layer directly bypasses another; all database access flows exclusively through the Application Layer, which enforces security controls before any query reaches the Data Layer.

```
┌──────────────────────────────────────────────────────────────────────┐
│                      PRESENTATION LAYER  (UI)                        │
│   EJS Templates: login.ejs · dashboard.ejs · search.ejs              │
│                  upload.ejs · admin.ejs · error.ejs                  │
│   Browser ─── HTTPS ─── Static Assets (/public)                      │
└──────────────────────────────────────────────────────────────────────┘
                                │  HTTP/Express
┌──────────────────────────────▼───────────────────────────────────────┐
│                SECURITY / MIDDLEWARE LAYER                           │
│  ┌──────────────┐  ┌────────────────┐  ┌──────────────────────┐     │
│  │  Helmet.js   │  │  Rate Limiter  │  │  isAuthenticated RBAC  │   │
│  │ (CSP, HSTS,  │  │  (5 req/15min) │  │  isAdmin middleware    |   |
│  │  X-Frame)    │  │                │  │                      │     │
│  └──────────────┘  └────────────────┘  └──────────────────────┘     │
│  ┌──────────────┐  ┌────────────────┐  ┌──────────────────────┐     │
│  │   Sessions   │  │  File Filter   │  │   Winston Audit Log  │     │
│  │ (HttpOnly,   │  │ (MIME + size)  │  │   + DB audit_log     │     │
│  │  1hr expiry) │  │                │  │                      │     │
│  └──────────────┘  └────────────────┘  └──────────────────────┘     │
└──────────────────────────────────────────────────────────────────────┘
                                │
┌──────────────────────────────▼───────────────────────────────────────┐
│                 APPLICATION LAYER  (server.js)                       │
│  Routes: GET/POST /login · /logout · /dashboard · /search            │
│          /search-vulnerable · /upload · /upload-vulnerable           │
│          /admin · /admin-vulnerable · /setup                         │
└──────────────────────────────────────────────────────────────────────┘
                                │  Parameterized SQL only
┌──────────────────────────────▼───────────────────────────────────────┐
│                    DATA LAYER  (database.js / SQLite)                │
│  Tables: users · grades · files · audit_log                          │
│  Controls: CHECK constraints · FOREIGN KEY · AUTOINCREMENT           │
└──────────────────────────────────────────────────────────────────────┘
```

The Security/Middleware Layer sits as an independent cross-cutting concern inside the Application Layer, implemented as Express middleware functions. This ensures that security checks (authentication,

authorization, logging) cannot be bypassed by adding new routes — they must explicitly be opted out of, which is never done for protected resources.

3. Required Threat Surface Implementation

All five required entry points are fully implemented with both vulnerable and secure counterparts:

Entry Point	Route(s)	Vulnerability Demonstrated	Secure Mitigation
Login Form	POST /login	Brute force, user enumeration	Rate limiter, bcrypt, generic errors
Search Form	GET /search-vulnerable GET /search	SQL Injection via string concat	Parameterized queries
File Upload	POST /upload-vulnerable POST /upload	Unrestricted upload (malware/DoS)	MIME allow-list, 1 MB cap, random names
Admin Panel	GET /admin-vulnerable GET /admin	Missing role check (BAC)	isAdmin RBAC middleware
REST APIs	All protected routes	Unauthenticated endpoint access	isAuthenticated middleware on every route

4. Authentication, Authorization & Session Management

4.1 Authentication

User passwords are hashed using **bcrypt** with a cost factor of 10 (bcryptjs library). Plaintext passwords are never stored or logged. On login, the hash is compared using `bcrypt.compare()` in constant time, preventing timing attacks.

- **Rate Limiting:** The `loginLimiter` middleware caps login attempts at 5 per 15-minute window per IP. Exceeding this triggers a 429 response and an audit log entry.
- **User Enumeration Prevention:** Both "user not found" and "wrong password" return the same generic message: "*Invalid username or password.*"
- **Credential Exposure:** Error messages never reveal which field was incorrect.

4.2 Authorization (RBAC)

Two middleware functions enforce access control:

- `isAuthenticated(req, res, next)` — Verifies `req.session.user` exists; redirects to `/login` otherwise.
- `isAdmin(req, res, next)` — Additionally verifies `req.session.user.role === 'admin'`; returns HTTP 403 and logs an unauthorized access attempt otherwise.

The vulnerable admin route (`/admin-vulnerable`) uses only `isAuthenticated`, demonstrating how a missing role check exposes sensitive data to any logged-in user (OWASP A01).

4.3 Session Management

- **HttpOnly cookie:** Prevents JavaScript access to the session cookie (mitigates XSS-based session theft).
- **SameSite:** Configured to prevent CSRF-style token theft.
- **Expiry:** Sessions expire after 1 hour (`maxAge: 3600000`).
- **Logout:** `req.session.destroy()` invalidates the server-side session immediately.
- **Session Secret:** Loaded from `process.env.SESSION_SECRET` (dotenv); a fallback development value is noted but not used in production.

5. Secure Coding Practices

5.1 Input Validation & Sanitization

- All search inputs are passed as parameters to the SQLite3 driver; string concatenation is never used in the secure route.
- Login fields are checked for presence before any database query is issued.
- File uploads are validated by MIME type (server-side, not just file extension) and file size.

5.2 Error Handling

- A global Express error handler (`app.use((err, req, res, next)`) catches all unhandled errors.
- Stack traces are logged internally (Winston) but **never sent to the client** — only a generic message is rendered via `error.ejs` .
- A custom 404 handler prevents default Express error messages from leaking framework details.

5.3 HTTP Security Headers (Helmet.js)

- **Content-Security-Policy (CSP):** `defaultSrc 'self'` — restricts resource loading to the same origin.
- **X-Frame-Options: SAMEORIGIN** — prevents clickjacking attacks.
- **X-Content-Type-Options: nosniff** — prevents MIME-type sniffing.
- **X-XSS-Protection** (legacy browsers): Enables browser-level XSS filter.

5.4 Output Encoding (XSS Prevention)

All dynamic values rendered in EJS templates use the `<%= %>` tag (HTML-encoded output), not the unescaped `<%- %>` . This ensures any user-supplied content is treated as text, not executable HTML/JavaScript.

6. Data Protection & Privacy Controls

6.1 Data in Transit

The application is configured to run over HTTP in development. The session cookie's `secure` flag is set to `false` for local development but should be set to `true` in production behind an HTTPS reverse proxy (e.g., NGINX with Let's Encrypt). Helmet's HSTS header enforces HTTPS once deployed.

6.2 Data at Rest

- **Passwords:** Stored exclusively as bcrypt hashes; original passwords are irrecoverable from the database.

- **Uploaded files:** Stored with randomly generated filenames (UUID-style, assigned by Multer) in the `/uploads` directory. Original filenames are retained only in the `files` database table, not in the file system path.
- **Audit Logs:** Stored in both the SQLite `audit_log` table and Winston file logs (`logs/combined.log` , `logs/error.log`) with timestamps in ISO 8601 format.

6.3 Data Minimization

Sessions store only the minimum required fields: `user.id` , `user.username` , and `user.role` . No password hash or sensitive PII is placed in the session or cookies. The `grades` table does not store student IDs or contact information beyond names, keeping the attack surface for data exposure minimal.

7. Database Design & Safety

7.1 Schema Overview

Table	Purpose	Key Columns	Safety Features
`users`	Credential & role storage	id, username (UNIQUE), password, role	CHECK constraint on role ∈ {admin, user}
`grades`	Student academic records	id, student_name, course, grade	NOT NULL on all columns
`files`	Upload metadata / audit trail	id, filename, path, uploaded_by	FOREIGN KEY → users.id
`audit_log`	Security event journal	id, timestamp (DEFAULT NOW), action, user	Append-only; DEFAULT CURRENT_TIMESTAMP

7.2 Safe Access Patterns

- All production routes use SQLite3's parameterized query API (`db.get(sql, [params])` , `db.all(sql, [params])` , `db.run(sql, [params])`).
- The only exception is the `/search-vulnerable` route, which intentionally uses string concatenation for demonstration and is labelled clearly in code and UI.
- The database connection is a single module (`database.js`) exported and reused — no connection strings are duplicated across the codebase.
- Foreign key enforcement ensures `files.uploaded_by` always references a valid `users.id` , preventing orphan records.

8. OWASP Top 10 Mapping

A01 – Broken Access Control

> **Threat:** Any authenticated user can access the admin panel without an admin role.
> **Demonstrated via:** `GET /admin-vulnerable` — no `isAdmin` check; any logged-in user sees all grade records.
> **Mitigation:** `GET /admin` applies the `isAdmin` middleware, which validates `req.session.user.role === 'admin'` and returns HTTP 403 + logs the unauthorized attempt if the check fails.
> **Evidence:** Login as *student1*, navigate to `/admin-vulnerable` (succeeds) vs. `/admin` (returns 403).

A03 – Injection (SQL Injection)

> **Threat:** User-controlled input is embedded directly into an SQL query string, allowing an attacker to manipulate query logic.
> **Demonstrated via:** `GET /search-vulnerable?q=' OR '1'='1` — the injected payload causes the query to return ALL rows from the grades table.
> **Mitigation:** `GET /search` uses placeholders: `db.all("SELECT ... WHERE student_name LIKE ?", [searchPattern])` . The same payload is treated as a literal string and returns no results.
> **Evidence:** Submit `' OR '1'='1` in both search forms and compare results.

A05 – Security Misconfiguration

> **Threats addressed:**
> - **Missing Security Headers:** Helmet.js configures CSP, X-Frame-Options, X-Content-Type-Options, and HSTS globally for all responses.
> - **Unrestricted File Upload:** `POST /upload-vulnerable` accepts any file type and size. `POST /upload` enforces a 1 MB size limit and MIME-type allow-list (PNG, JPEG, PDF).
> - **Information Disclosure:** The global error handler suppresses stack traces; the 404 handler prevents Express default pages from leaking version information.
> - **Default Credentials / Setup Route:** The `/setup` route is documented as requiring removal in production.

A07 – Identification & Authentication Failures

> **Threats addressed:**
> - **Weak Password Storage:** bcrypt (cost=10) is used; passwords are never stored plaintext.
> - **Brute Force:** `loginLimiter` limits to 5 attempts per 15-minute window; excessive attempts are logged and trigger HTTP 429.
> - **User Enumeration:** Both "user not found" and "wrong password" produce identical error text.
> - **Session Fixation:** A new session is initialized on successful login; old session data is regenerated.
> - **Session Expiry:** Sessions have a 1-hour TTL; logout calls `session.destroy()` .

A09 – Security Logging & Monitoring Failures

> **Threat:** Without adequate logging, security incidents go undetected and non-repudiation is impossible.
> **Mitigation — Dual Logging System:**
> - **Winston:** Structured JSON logs written to `logs/combined.log` (all events) and `logs/error.log` (errors only), plus console during development.
> - **Database audit_log:** Every security-relevant event (login success/failure, admin access, rate limit exceeded, file uploads, unauthorized attempts) is inserted into the `audit_log` table with timestamp and username via the `auditLog()` function.
>
> **Events Logged:** Login success, login failure, logout, rate limit exceeded, search queries, file uploads (both modes), admin panel access, unauthorized access attempts, server start.

9. STRIDE Threat Model

The following table maps each STRIDE threat category to specific entry points in the application, describes the risk, and documents the implemented mitigation.

Type	Entry Point	Risk Description	Mitigation
S Spoofing	Login Form `POST /login`	Attacker impersonates a legitimate user by guessing or brute-forcing credentials. Credential stuffing using leaked password lists is a realistic attack vector.	bcrypt hashing (cost=10) makes offline attacks slow. Rate limiter (5 req/15 min) limits online guessing. Generic error messages prevent user enumeration.
S Spoofing	Session Cookie (all authenticated routes)	XSS vulnerability in another page could allow an attacker to steal the session cookie and replay it to impersonate the authenticated user.	HttpOnly cookie flag prevents JavaScript access. EJS output encoding (`<%=`) prevents XSS insertion. 1-hour session TTL limits exposure window.
T Tampering	Search Form `GET /search-vulnerable`	SQL injection via string concatenation allows an attacker to manipulate the SQL query, extracting all grade records or potentially modifying data using stacked queries.	Secure route (`/search`) uses parameterized queries via SQLite3's placeholder API. User input is never interpreted as SQL code.
T Tampering	File Upload `POST /upload-vulnerable`	Attacker uploads a PHP/JS web shell or overwrites server-side files. Executing a web shell grants full server compromise. Large files (DoS) can fill disk and crash the server.	Secure route validates MIME type server-side (not just extension), enforces 1 MB size cap, and stores files with random Multer-assigned names — no original name in path.
T Tampering	HTTP Response Headers (all routes)	Without security headers, an attacker can inject scripts (XSS), load the app in an iframe (clickjacking), or trick browsers into executing uploaded files as HTML (MIME sniffing).	Helmet.js applies CSP (`defaultSrc 'self'`), X-Frame-Options (SAMEORIGIN), X-Content-Type-Options (nosniff), and Referrer-Policy globally on all responses.
R Repudiation	All Authenticated	A user performs a malicious action (unauthorized admin access	Every security-relevant event calls `auditLog(action, user)` , writing

Type	Entry Point	Risk Description	Mitigation
	Routes	attempt, SQL injection) and later denies it. Without audit trails, there is no evidence to attribute the action.	to both the database `audit_log` table (with timestamp) and Winston log files. Logs include username and timestamp for non-repudiation.
I Info Disclosure	Admin Panel `GET /admin-vulnerable`	A student-role user navigates directly to `/admin-vulnerable` and views the complete grade table for all students — information they are not authorized to see.	Secure route (`/admin`) applies `isAdmin` middleware. Users without `role = 'admin'` receive HTTP 403. The unauthorized access is logged to audit trail.
I Info Disclosure	Error Pages & Responses	Unhandled exceptions that leak stack traces expose file paths, library versions, and internal logic that an attacker can use to craft targeted exploits.	Global Express error handler catches all unhandled errors, logs the full stack internally (Winston), but renders only a generic "Something went wrong" message to the client via `error.ejs` .
I Info Disclosure	Login Form `POST /login`	Differential error messages ("user not found" vs. "wrong password") allow an attacker to enumerate valid usernames by observing response content.	Both failure conditions return identical text: "*Invalid username or password.*" Response timing is also equalized by always running bcrypt.compare (even for non-existent users in production-grade implementations).
D Denial of Service	Login Form `POST /login`	An automated attacker floods the login endpoint with thousands of requests, consuming server CPU (bcrypt is intentionally expensive) and potentially blocking legitimate users.	`express-rate-limit` restricts login attempts to 5 per 15 minutes per IP. Excess requests receive HTTP 429 without invoking bcrypt, preserving server resources.
D Denial of Service	File Upload `POST /upload-vulnerable`	Attacker uploads a multi-gigabyte file (e.g., a fork bomb or large binary), filling the server's disk or exhausting memory buffers, causing service unavailability.	Secure upload route applies Multer's `limits.fileSize: 1048576` (1 MB). Files exceeding this trigger a `LIMIT_FILE_SIZE` error before data is written to disk.
E Elevation of Privilege	Admin Panel `GET /admin-vulnerable`	A student-role user directly accesses the admin URL, effectively gaining admin-level data access without authorization — a classic horizontal privilege escalation scenario.	`isAdmin` middleware checks `req.session.user.role === 'admin'` before granting access to `/admin` . Role is stored server-side in the session, not in a client-side cookie or header that could be manipulated.
E Elevation of Privilege	Session / Cookie	If a user could modify their session cookie to change their `role` to `'admin'` , they could bypass all RBAC controls. Client-side session storage (e.g., JWT without signature verification) would enable this.	Sessions are stored server-side using `express-session` . The client holds only an opaque session ID cookie (HttpOnly). Tampering with the cookie value invalidates the session lookup.

10. Security Testing

Security testing was conducted manually throughout development using the application's dual-mode design. Each vulnerability was first demonstrated in the vulnerable version, then verified as mitigated in the secure version.

Test Case	Method	Expected Vulnerable Result	Expected Secure Result	Status
SQL Injection via Search	Input: `' OR '1'='1`	All grade rows returned	Zero results returned	**PASS**
Admin Access – Student User	Login as student1, visit /admin	/admin-vulnerable: data visible	/admin: HTTP 403 returned	**PASS**
Brute Force Login	Submit 6 failed logins	All 6 attempts processed	6th attempt returns HTTP 429	**PASS**
Malicious File Upload (.exe)	Upload Windows executable	/upload-vulnerable: accepted	/upload: "Invalid file type" error	**PASS**
Oversized File Upload	Upload 5 MB file	/upload-vulnerable: accepted	/upload: "File too large" error	**PASS**
XSS in Search	Input: `<script>alert(1)</script>`	Script executes in browser	HTML encoded, displayed as text	**PASS**
Unauthenticated Access	Visit /dashboard without login	—	Redirected to /login	**PASS**
Stack Trace Disclosure	Trigger server error	Stack trace in browser	Generic error message only	**PASS**
Security Headers Present	Inspect HTTP response headers	—	CSP, X-Frame-Options present	**PASS**
Audit Log Written	Perform login, view audit_log	—	Event recorded with user + timestamp	**PASS**

11. Deployment Security & Incident Response

11.1 Deployment Recommendations

- **HTTPS:** Deploy behind an NGINX reverse proxy with Let's Encrypt TLS. Set `cookie.secure = true` in production.
- **Environment Variables:** All secrets (`SESSION_SECRET`) must be loaded from environment variables, never hard-coded. Use a `.env` file only in development (add to `.gitignore`).
- **Remove /setup route:** The database seeding route must be disabled or removed before production deployment.
- **Process Manager:** Use PM2 or systemd to run the Node.js process, ensuring automatic restart on crash.

- **Log Rotation:** Configure Winston or system log rotation to prevent disk exhaustion from audit logs.
- **Principle of Least Privilege:** Run the Node.js process as a non-root user with minimal filesystem permissions.

11.2 Backup & Recovery

- The SQLite database file (`student.db`) should be backed up daily to an off-site location.
- Uploaded files (`/uploads`) should be included in backup scope.
- Log files should be archived with a 90-day retention policy per typical compliance requirements.

11.3 Incident Response Procedures

- **Detection:** Monitor `logs/combined.log` for patterns: repeated failed logins, rate limit triggers, unauthorized admin access attempts.
- **Containment:** Invalidate all active sessions by restarting the server (session store is in-memory). Optionally block offending IPs at the firewall level.
- **Analysis:** Query the `audit_log` table to reconstruct the timeline of attacker actions.
- **Recovery:** Restore the database from the most recent clean backup. Reset compromised credentials (bcrypt rehash with new password).
- **Post-Incident:** Patch the vulnerability, update documentation, and review all related controls.

12. Conclusion

The Secure Student Portal successfully demonstrates five OWASP Top 10 categories through a dual-mode architecture that allows direct, side-by-side comparison of vulnerable and secure implementations. The layered architecture enforces strict separation of concerns, ensuring security middleware cannot be bypassed by adding new routes without explicitly removing protections.

The STRIDE threat model analysis identified twelve distinct threat scenarios across the application's five entry points. All twelve threats have documented mitigations implemented in the codebase. The security testing section confirms all mitigations behave as expected under manual testing.

The project demonstrates that security does not require complexity — clear architectural boundaries, consistent use of well-maintained libraries (bcrypt, Helmet, express-rate-limit), and disciplined use of parameterized queries are sufficient to eliminate the most critical web application vulnerabilities.